

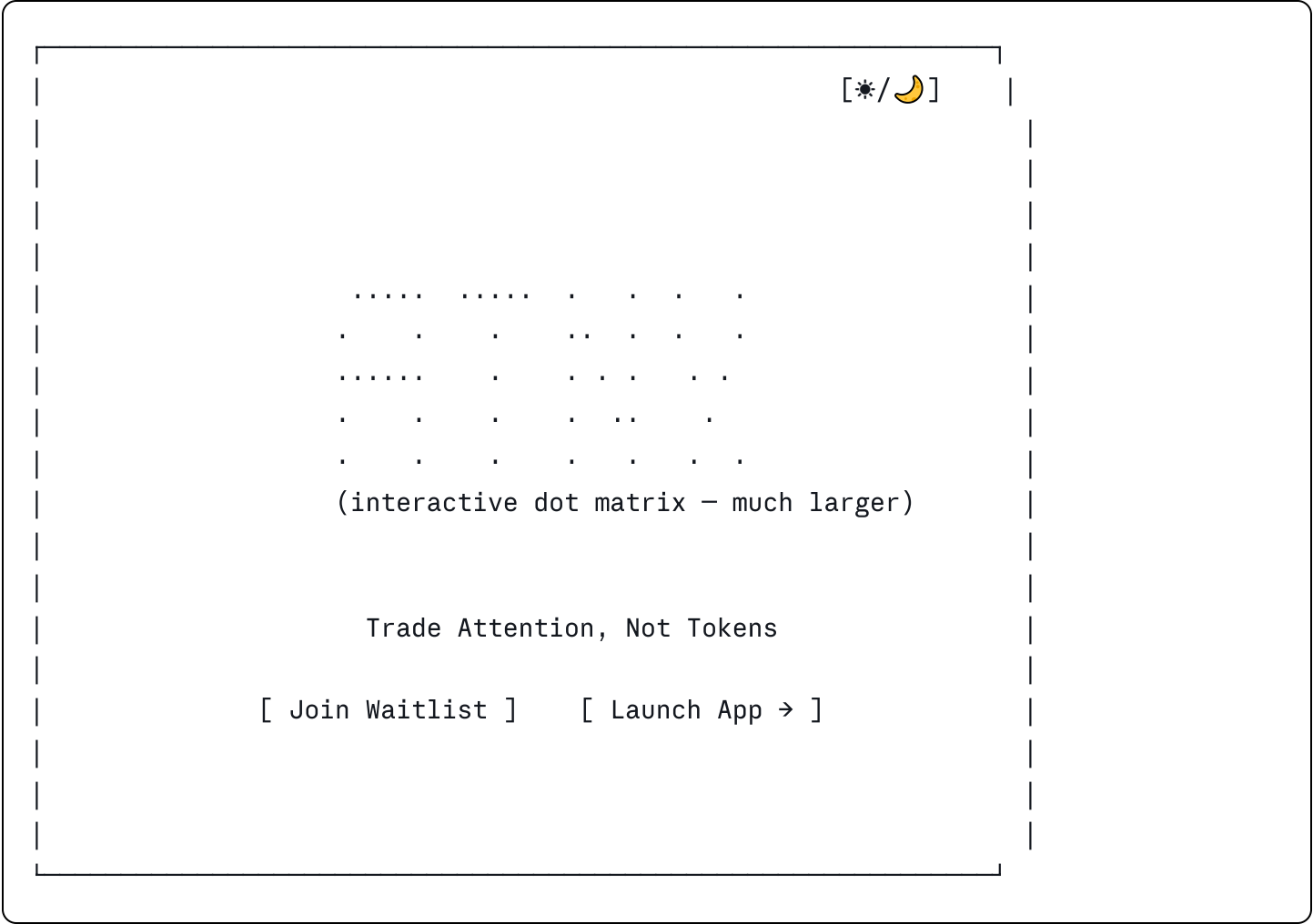
ATNX Landing Page — Interactive Dot Matrix Logo

Overview

Build a minimal, high-impact landing page for ATNX. The page has one hero element: a large interactive dot matrix logo spelling "ATNX" in CMYK colors on a black background. Below it, a tagline and two CTA buttons. That's it. No scrolling sections, no complex animations. Clean, confident, memorable.

The page includes the dark/light mode toggle from the main app.

Page Structure



The entire page is one viewport height (100vh). No scroll. Everything centered.

Tech Stack

- **Canvas2D** for the dot matrix (NOT Three.js — simpler, lighter, more reliable)

- **Next.js** page route at `/` (replace or restructure current dashboard)
 - **next-themes** for dark/light mode (already installed from rebrand)
 - **Tailwind CSS** for layout and buttons
-

Route Structure Update

```
/           - Landing page (NEW - this page)
/app        - Dashboard (move existing dashboard here)
/app/portfolio - Portfolio page (move existing portfolio here)
```

The "Launch App" button on the landing page links to `/app`.

Part 1: Dot Matrix Canvas

How It Works

1. Create a hidden offscreen canvas
2. Draw the text "ATNX" on it in a large bold font
3. Read the pixel data from that canvas
4. For every filled pixel, create a dot/particle at that position
5. Render those dots on the visible canvas
6. Each letter gets its own color based on CMYK mapping
7. Mouse interaction: dots near the cursor scatter away, then drift back

Letter Color Mapping

```
typescript
```

```
const LETTER_COLORS = {  
  dark: {    // Dark mode (black background)  
    A: '#00D4FF',    // Cyan  
    T: '#FF00E5',    // Magenta  
    N: '#FFE500',    // Yellow  
    X: '#FFFFFF',    // White (contrast on black – represents K)  
  },  
  light: {   // Light mode (white background)  
    A: '#00B8DB',    // Cyan (slightly darker for light bg)  
    T: '#D900C5',    // Magenta (slightly darker for light bg)  
    N: '#D4BE00',    // Yellow (slightly darker for light bg)  
    X: '#000000',    // Black (true K – contrast on white)  
  },  
};
```

To assign colors per letter, you need to know which letter each dot belongs to. When sampling pixels from the offscreen canvas, track the x-position boundaries of each letter:

typescript

```
// After drawing "ATNX" on offscreen canvas, measure each letter's width
// using ctx.measureText() for each letter individually
// Then when sampling pixels, determine which letter region the x coordinate falls in

interface LetterBounds {
  letter: string;
  startX: number;
  endX: number;
  color: string;
}

// Calculate bounds for each letter
function getLetterBounds(ctx: CanvasRenderingContext2D, text: string, startX: number) {
  const bounds: LetterBounds[] = [];
  let currentX = startX;

  for (const char of text) {
    const metrics = ctx.measureText(char);
    bounds.push({
      letter: char,
      startX: currentX,
      endX: currentX + metrics.width,
      color: '', // assigned based on theme
    });
    currentX += metrics.width;
  }

  return bounds;
}
```

Dot/Particle Structure

typescript

```
interface Dot {  
  // Original position (where it belongs in the text)  
  originX: number;  
  originY: number;  
  
  // Current position (may be displaced by mouse)  
  x: number;  
  y: number;  
  
  // Velocity (for physics)  
  vx: number;  
  vy: number;  
  
  // Visual  
  color: string;  
  size: number;      // Dot radius – vary slightly for organic feel (1.5 to 3px)  
  opacity: number;   // Can pulse or vary  
}
```

Sampling Pixels to Create Dots

typescript

```

function createDotsFromText(
  text: string,
  fontSize: number,
  canvasWidth: number,
  canvasHeight: number,
  theme: 'dark' | 'light'
): Dot[] {
  // 1. Create offscreen canvas
  const offscreen = document.createElement('canvas');
  offscreen.width = canvasWidth;
  offscreen.height = canvasHeight;
  const ctx = offscreen.getContext('2d')!;

  // 2. Draw text centered
  ctx.fillStyle = '#FFFFFF';
  ctx.font = `bold ${fontSize}px "JetBrains Mono", monospace`;
  ctx.textAlign = 'center';
  ctx.textBaseline = 'middle';
  ctx.fillText(text, canvasWidth / 2, canvasHeight / 2);

  // 3. Get letter bounds for color assignment
  // Need to calculate starting X for centered text
  const totalWidth = ctx.measureText(text).width;
  const textStartX = (canvasWidth - totalWidth) / 2;
  const letterBounds = getLetterBounds(ctx, text, textStartX);

  // Assign colors based on theme
  const colors = LETTER_COLORS[theme];
  letterBounds[0].color = colors.A;
  letterBounds[1].color = colors.T;
  letterBounds[2].color = colors.N;
  letterBounds[3].color = colors.X;

  // 4. Sample pixels
  const imageData = ctx.getImageData(0, 0, canvasWidth, canvasHeight);
  const dots: Dot[] = [];
  const gap = 4; // Sample every 4th pixel - adjust for density vs performance

  for (let y = 0; y < canvasHeight; y += gap) {
    for (let x = 0; x < canvasWidth; x += gap) {
      const index = (y * canvasWidth + x) * 4;
      const alpha = imageData.data[index + 3];
    }
  }
}

```

```
if (alpha > 128) {
  // This pixel is part of the text – determine which letter
  const letterBound = letterBounds.find(b => x >= b.startX && x < b.endX);
  const color = letterBound?.color || colors.A;

  dots.push({
    originX: x,
    originY: y,
    x: x,
    y: y,
    vx: 0,
    vy: 0,
    color: color,
    size: 1.5 + Math.random() * 1.5, // 1.5 to 3px
    opacity: 0.8 + Math.random() * 0.2,
  });
}
}
}

return dots;
}
```

Mouse Interaction Physics

typescript

```

const MOUSE_RADIUS = 80;           // How far the mouse influence reaches (pixels)
const SCATTER_FORCE = 8;           // How hard dots push away
const RETURN_SPEED = 0.08;         // How fast dots drift back (0-1, lower = slower/smooth)
const FRICTION = 0.85;             // Velocity damping per frame

function updateDots(dots: Dot[], mouseX: number, mouseY: number) {
  for (const dot of dots) {
    // Calculate distance to mouse
    const dx = dot.x - mouseX;
    const dy = dot.y - mouseY;
    const distance = Math.sqrt(dx * dx + dy * dy);

    // If mouse is within radius, apply repulsion force
    if (distance < MOUSE_RADIUS && distance > 0) {
      const force = (MOUSE_RADIUS - distance) / MOUSE_RADIUS;
      const angle = Math.atan2(dy, dx);
      dot.vx += Math.cos(angle) * force * SCATTER_FORCE;
      dot.vy += Math.sin(angle) * force * SCATTER_FORCE;
    }

    // Apply velocity
    dot.x += dot.vx;
    dot.y += dot.vy;

    // Apply friction
    dot.vx *= FRICTION;
    dot.vy *= FRICTION;

    // Drift back to original position
    dot.x += (dot.originX - dot.x) * RETURN_SPEED;
    dot.y += (dot.originY - dot.y) * RETURN_SPEED;
  }
}

```

Render Loop

typescript

```
function render(ctx: CanvasRenderingContext2D, dots: Dot[]) {
  ctx.clearRect(0, 0, ctx.canvas.width, ctx.canvas.height);

  for (const dot of dots) {
    ctx.beginPath();
    ctx.arc(dot.x, dot.y, dot.size, 0, Math.PI * 2);
    ctx.fillStyle = dot.color;
    ctx.globalAlpha = dot.opacity;
    ctx.fill();
  }

  ctx.globalAlpha = 1;
}
```

Animation Loop

Use requestAnimationFrame for smooth 60fps:

```
typescript

function animate() {
  updateDots(dots, mouseX, mouseY);
  render(ctx, dots);
  requestAnimationFrame(animate);
}
```

Canvas Sizing

The canvas should be large and centered. Size it to fill most of the viewport width but leave room for the tagline and buttons below:

```
typescript

// Canvas dimensions
const canvasWidth = Math.min(window.innerWidth * 0.85, 1200); // max 1200px
const canvasHeight = canvasWidth * 0.3; // Aspect ratio for "ATNX" text

// Font size scales with canvas
const fontSize = canvasHeight * 0.7;
```

Handle window resize — recalculate canvas size and regenerate dots.

Mouse Tracking

typescript

```
canvas.addEventListener('mousemove', (e) => {
  const rect = canvas.getBoundingClientRect();
  mouseX = e.clientX - rect.left;
  mouseY = e.clientY - rect.top;
});

// When mouse leaves canvas, reset position to off-screen so dots settle
canvas.addEventListener('mouseleave', () => {
  mouseX = -1000;
  mouseY = -1000;
});
```

Touch Support

For mobile, use touch events:

typescript

```
canvas.addEventListener('touchmove', (e) => {
  e.preventDefault();
  const touch = e.touches[0];
  const rect = canvas.getBoundingClientRect();
  mouseX = touch.clientX - rect.left;
  mouseY = touch.clientY - rect.top;
});

canvas.addEventListener('touchend', () => {
  mouseX = -1000;
  mouseY = -1000;
});
```

Part 2: Theme Integration

When the theme toggles between dark and light:

1. Background changes: black (● #0A0A0A) ↔ white (□ #FAFAFA)
2. The X letter dots change color: white (□ #FFFFFF) ↔ black (● #000000)
3. Cyan, Magenta, Yellow dots shift to their light-mode-safe variants

4. Tagline text color inverts
5. Button styles adapt

When theme changes, regenerate the dot colors (don't recreate dots — just update the color property):

typescript

```
function updateDotColors(dots: Dot[], letterBounds: LetterBounds[], theme: 'dark' |  
  const colors = LETTER_COLORS[theme];  
  
  // Re-assign colors to letter bounds  
  letterBounds[0].color = colors.A;  
  letterBounds[1].color = colors.T;  
  letterBounds[2].color = colors.N;  
  letterBounds[3].color = colors.X;  
  
  // Update each dot's color based on its origin position  
  for (const dot of dots) {  
    const letterBound = letterBounds.find(b => dot.originX >= b.startX && dot.origin  
    if (letterBound) {  
      dot.color = letterBound.color;  
    }  
  }  
}
```

Use `useTheme()` from next-themes to detect changes and trigger the color update.

Part 3: Background Floating Particles

Add a sparse layer of slowly drifting particles behind the dot matrix logo. These are NOT part of the text — they're ambient atmosphere.

typescript

```

interface BackgroundParticle {
  x: number;
  y: number;
  vx: number;
  vy: number;
  color: string;
  size: number;
  opacity: number;
}

function createBackgroundParticles(count: number, width: number, height: number, theme: string) {
  const colors = [
    LETTER_COLORS[theme].A, // Cyan
    LETTER_COLORS[theme].T, // Magenta
    LETTER_COLORS[theme].N, // Yellow
  ];

  return Array.from({ length: count }, () => ({
    x: Math.random() * width,
    y: Math.random() * height,
    vx: (Math.random() - 0.5) * 0.3, // Very slow drift
    vy: (Math.random() - 0.5) * 0.3,
    color: colors[Math.floor(Math.random() * 3)],
    size: 1 + Math.random() * 2,
    opacity: 0.1 + Math.random() * 0.2, // Very subtle
  }));
}

```

- About 40-60 particles total — sparse, not distracting
- They drift slowly in random directions
- Wrap around screen edges (if particle goes off right, appear on left)
- Very low opacity (0.1-0.3) — they're atmosphere, not content
- Render on the same canvas BEFORE the dot matrix text (so text dots are on top)
- These particles are NOT affected by mouse interaction

Part 4: Tagline & Buttons

Below the canvas, centered:

Tagline

Trade Attention, Not Tokens

- Font: Inter (sans-serif), 400 weight
- Size: text-xl on mobile, text-2xl on desktop
- Color: text-secondary (gray that works on both dark and light)
- Subtle letter-spacing: 0.05em
- Margin top: ~40px below the canvas

Subtitle (optional, smaller)

The Attention Economy Exchange

- Font: JetBrains Mono, 400 weight
- Size: text-sm
- Color: text-tertiary
- Below the tagline with small gap

Buttons

Two buttons, side by side, centered:

Join Waitlist:

- Border: 1px solid, cyan color
- Background: transparent
- Text: cyan color, JetBrains Mono
- Hover: cyan background, black text
- Padding: 12px 32px
- Border radius: 8px

Launch App →:

- Border: none
- Background: magenta (solid fill)
- Text: black (#0A0A0A), JetBrains Mono, bold
- Hover: magenta-dim (slightly darker), subtle magenta glow
- Padding: 12px 32px
- Border radius: 8px
- The → arrow is part of the text

Button gap: ~16px between them.

In light mode, "Launch App" text stays black (works on magenta). "Join Waitlist" uses the light-mode cyan variant.

Waitlist Button Behavior

For MVP, clicking "Join Waitlist" can either:

Option A (simplest): Open a mailto link or a Google Form in a new tab.

Option B (slightly better): Show a small inline email input that expands from the button:

```
[ Join Waitlist ] → clicks → [ your@email.com [Submit] ]
```

Store emails in localStorage for now or POST to an API route. Don't over-engineer this — a simple email collection is fine for MVP.

Option C (best if time allows): A small modal with email input, styled in the CMYK theme. On submit, show a confirmation: "You're on the list ✓" in cyan.

Go with whichever is fastest to implement. Option A is fine.

Launch App Button

Links to `/app` — the main dashboard.

Part 5: Theme Toggle on Landing Page

Place the dark/light mode toggle in the top-right corner of the page:

```
Position: fixed, top: 24px, right: 24px
```

Same toggle component from the main app. When toggled:

- Background transitions smoothly (black ↔ white)

- Dot colors update (especially X: white ↔ black)
 - Text and button colors adapt
 - Background particles shift to light-mode color variants
-

Part 6: Responsive Design

Desktop (>1024px)

- Canvas: up to 1200px wide
- Font size: large (~180-240px on offscreen canvas)
- Full dot density (gap = 4)
- Mouse interaction active

Tablet (768-1024px)

- Canvas: 90% viewport width
- Font size: scales down
- Slightly reduced dot density (gap = 5)
- Touch interaction active

Mobile (<768px)

- Canvas: 95% viewport width
 - Font size: scales down significantly
 - Reduced dot density (gap = 6) for performance
 - Touch interaction active
 - Buttons stack vertically instead of side by side
 - Tagline text size: text-lg
-

Component Structure

```
app/  
├─ page.tsx           # Landing page (NEW)  
├─ app/  
│   ├─ page.tsx       # Dashboard (moved from /)  
│   └─ portfolio/
```

```
|      └─ page.tsx          # Portfolio (moved)
└─ components/
    └─ DotMatrixLogo.tsx    # The canvas component
```

DotMatrixLogo.tsx

Single self-contained React component that:

- Creates and manages the canvas
- Handles dot generation from text
- Runs the animation loop
- Handles mouse/touch events
- Responds to theme changes
- Handles resize events
- Cleans up on unmount (cancel animation frame, remove listeners)

tsx

```

'use client';

import { useRef, useEffect } from 'react';
import { useTheme } from 'next-themes';

export function DotMatrixLogo() {
  const canvasRef = useRef<HTMLCanvasElement>(null);
  const { theme } = useTheme();

  useEffect(() => {
    const canvas = canvasRef.current;
    if (!canvas) return;

    // Initialize everything here
    // Set up dots, mouse tracking, animation loop
    // Return cleanup function

    return () => {
      // Cancel animation frame
      // Remove event listeners
    };
  }, [theme]); // Re-initialize when theme changes

  return (
    <canvas
      ref={canvasRef}
      className="block mx-auto"
      // Width and height set programmatically in useEffect
    />
  );
}

```

page.tsx (Landing Page)

```
tsx
```

```

import { DotMatrixLogo } from '@components/DotMatrixLogo';
import { ThemeToggle } from '@components/ThemeToggle';

export default function LandingPage() {
  return (
    <main className="h-screen w-screen flex flex-col items-center justify-center
      bg-dark-primary dark:bg-dark-primary light:bg-light-primary
      overflow-hidden relative">

      {/* Theme toggle – fixed top right */}
      <div className="fixed top-6 right-6 z-50">
        <ThemeToggle />
      </div>

      {/* Dot matrix logo */}
      <DotMatrixLogo />

      {/* Tagline */}
      <p className="mt-10 text-2xl font-sans text-gray-500 tracking-wide">
        Trade Attention, Not Tokens
      </p>

      {/* Subtitle */}
      <p className="mt-2 text-sm font-mono text-gray-600">
        The Attention Economy Exchange
      </p>

      {/* CTA Buttons */}
      <div className="mt-10 flex gap-4">
        <button className="...waitlist styles...">
          Join Waitlist
        </button>
        <a href="/app" className="...launch styles...">
          Launch App →
        </a>
      </div>
    </main>
  );
}

```

- **Dot count matters.** At gap=4, "ATNX" in a 1200px wide canvas might generate 2000-4000 dots. This is fine for 60fps on most devices. If performance is an issue, increase the gap to 5 or 6.
- **Use requestAnimationFrame**, not setInterval. This ensures smooth rendering synced to the display refresh rate.
- **Don't create new objects in the render loop.** Pre-allocate all dots and mutate in place.
- **Canvas resolution:** For sharp rendering on retina displays, set canvas width/height to 2x the CSS size and scale the context: `ctx.scale(2, 2)`. Set CSS width/height via style attribute.

typescript

```
const dpr = window.devicePixelRatio || 1;
canvas.width = canvasWidth * dpr;
canvas.height = canvasHeight * dpr;
canvas.style.width = `${canvasWidth}px`;
canvas.style.height = `${canvasHeight}px`;
ctx.scale(dpr, dpr);
```

Implementation Priority

1. **Route restructure** — move dashboard to /app, create new landing page at /
2. **DotMatrixLogo component** — canvas setup, text sampling, dot creation
3. **Render loop** — draw dots on canvas at 60fps
4. **Mouse interaction** — scatter and return physics
5. **Letter coloring** — CMYK per letter
6. **Theme integration** — color switching on dark/light toggle
7. **Background particles** — ambient floating dots
8. **Tagline and buttons** — styled with CMYK theme
9. **Responsive sizing** — canvas and dot density adapt to screen size
10. **Touch support** — mobile interaction
11. **Waitlist functionality** — email collection (simplest approach that works)

Success Criteria

The landing page is done when:

1. ☒ Large "ATNX" displayed as interactive dot matrix, centered on screen
2. ☒ Each letter has its own CMYK color (A=cyan, T=magenta, N=yellow, X=white/black)
3. ☒ Dots scatter away from cursor and drift back smoothly
4. ☒ Touch interaction works on mobile
5. ☒ Dark/light mode toggle works, X letter switches white↔black
6. ☒ Subtle background particles float in CMYK colors
7. ☒ Tagline "Trade Attention, Not Tokens" displayed below logo
8. ☒ "Join Waitlist" button (cyan, outlined) is functional
9. ☒ "Launch App →" button (magenta, filled) links to /app
10. ☒ Page is responsive across desktop, tablet, and mobile
11. ☒ Smooth 60fps animation with no jank
12. ☒ Existing app functionality is preserved at /app and /app/portfolio